



Auto Marketplace Modernization

Complete Technical Project Overview

Laravel 13

PHP 8.3

Next.js 16

React 19

TypeScript 5

PostgreSQL

Tailwind CSS 4

Sanctum

Generated 25 June 2026 • Monorepo • Early / MVP Stage

A greenfield, modern **monorepo** rebuild of an auto marketplace platform, developed locally under XAMPP (`xampp/htdocs/`). Despite the name "modernization," there is **no legacy PHP code being migrated** — this is a clean-slate build on a current-generation stack. Work began ~16 June 2026 and is in **early / MVP stage**.

1. High-Level Architecture

A decoupled **API-first** architecture with three independent applications:

```
auto-marketplace-modernization/  
├─ backend/laravel/   → Laravel 13 REST API (data + business logic)  
├─ frontend/          → Next.js 16 customer-facing marketplace  
├─ admin/             → Next.js 16 dealer/admin dashboard (scaffold only)  
├─ docs/              → Schema & module planning docs  
├─ infrastructure/    → (empty – reserved for deploy config)  
└─ scripts/           → (empty – reserved for utilities)
```

- **Backend** serves JSON over `http://127.0.0.1:8001/api`
- **Frontend** consumes it via Axios / native `fetch`
- No Docker, no CI/CD yet — local Apache/PHP + `next dev`

2. Technology Stack

Layer	Technology	Version / Notes
Backend framework	Laravel	13.8
Language	PHP	8.3+
Authentication	Laravel Sanctum	4.0 (configured, not yet enforced)
Permissions	spatie/laravel-permission	8.0 (installed, unused)
Cache / Queue	Redis (predis) + database driver	3.5
Database	PostgreSQL (recently migrated from MySQL)	port 5432
Frontend / Admin	Next.js	16.2.9
UI runtime	React	19.2.4
Language	TypeScript	5 (strict mode)
Styling	Tailwind CSS	4
HTTP client	Axios	1.18
Dev tooling	Pint, PHPUnit 12, Pail, Faker, ESLint 9	—

 Next.js 16 is flagged in `AGENTS.md` as having **breaking changes** vs. older conventions — the bundled docs in `node_modules/next/dist/docs/` are the source of truth for that app.

3. Database Design

8 migrations, 5 domain tables + Laravel system tables (users, sessions, cache, jobs, personal_access_tokens). Uses **UUIDs, slugs, soft deletes**, and proper **foreign keys**.



Table	Key Columns & Notes
dealers	uuid, name, slug, email, phone, website, address, city, country, logo, is_active · <i>soft deletes</i>
inventory_sources	uuid, name, slug, type, base_url, api_key/api_secret, last_synced_at, sync_interval (def 60), is_active · <i>soft deletes</i> — <i>infra for external feed syncing, not yet implemented</i>
vehicles	uuid, slug, title, description, vin (unique), make, model, year, price decimal(12,2), currency (USD), mileage, fuel_type, transmission, condition, body_type, color, dealer_id (FK nullOnDelete), inventory_source_id (FK nullOnDelete), status enum (draft/active/pending/sold/archived), published_at · <i>soft deletes</i>
vehicle_images	vehicle_id (FK cascadeOnDelete), image_path, alt_text, image_source, is_featured, sort_order

Seed data (DatabaseSeeder via Faker factories): 10 dealers, 5 inventory sources, 100 vehicles, ~300 images (2–5 per vehicle).

4. Backend API (Laravel)

Currently minimal — one working endpoint:

Method	Route	Action	Status
GET	/api/vehicles	VehicleController@index	✅ Working — filters by make, model, year; eager-loads dealer + images; paginates 12/page
GET	/api/vehicles/{vehicle}	show	⚠️ show() not implemented

- **Output transformation:** `VehicleResource` exposes a curated JSON shape (id, uuid, title, slug, make/model/year, price, mileage, dealer *name only*, mapped `images` array).
- **Models:** Vehicle, Dealer, VehicleImage, InventorySource, User — with HasFactory, SoftDeletes, and Eloquent relationships.
- **Not yet built:** Form Request validation, Services, Jobs, Middleware, inventory-sync logic, auth-protected routes.

5. Frontend (Next.js Customer App)

Two routes, lean MVP:

- **/ (home)** — Client component. Live search/filter by **make** and **year** (re-fetches via `useEffect` on input change), renders clickable vehicle cards linking to detail pages.
- **/vehicles/[id] (detail)** — Server component using native `fetch` with `cache: "no-store"`. Shows full specs + a **3-column image gallery**.
- **API layer:** single Axios instance in `src/services/api.ts` (baseUrl `http://127.0.0.1:8001/api`).
- **State / data:** React hooks only — no Redux/Zustand, no React Query/SWR.
- **Styling:** Tailwind 4 with light/dark CSS variables; Geist fonts.
- **TypeScript:** strict mode on, but domain types are loose (`any[]`) — no shared Vehicle interface yet.

Admin app: scaffold only (layout + home), same stack.

6. Current Work-in-Progress (uncommitted)

Two files modified vs. last commit (`9c402fe` Setup PostgreSQL and vehicle details image gallery):

- `VehicleController.php` — added a **model** filter alongside make/year.
- `frontend/src/app/page.tsx` — wired the listing/search UI to `/vehicles` with make + year params.

7. Development Timeline

16 commits showing clear progression: monorepo scaffold → architecture & DB planning → vehicle/dealer/inventory schema iterations → models & migrations → factories/seeder → backend↔frontend API integration → **MySQL→PostgreSQL migration** + image gallery (most recent).

8. Notable Gaps / Next Steps

The plan (`docs/DATABASE_PLAN.md`) envisions **10 modules**; only ~4 are partially built. Still to come:

- **Authentication & roles** — Sanctum + spatie/permission installed but unused; no login/protected endpoints.
- **Vehicle show endpoint** — referenced but not implemented.
- **Inventory sync** — table ready, but no Jobs/Services pull external feeds.
- **Favorites, advanced search, SEO, blogs** — planned, not started.
- **Shared TS types**, form validation, admin dashboard build-out.
- **Infra:** no Docker, no CI/CD; `.env` with a real DB password is committed (⚠️ should be removed from version control).

Summary — A cleanly-architected, modern monorepo (Laravel 13 API + Next.js 16 / React 19 + PostgreSQL) for a vehicle marketplace. The data model is well-designed and seeded; the read path (list + filter + detail + image gallery) works end-to-end. It is an early-stage MVP — the foundation is solid, but auth, write operations, inventory syncing, and most secondary modules remain to be built.